

OPERATIONS & USER GUIDE

MCS

Merchant Checkout System

A practical guide for merchants, customers, administrators, and demo reviewers

Edition	Scope	Status
1.0 · 14 July 2026	BRD end-to-end processes	Verified build

What this guide covers

Account access, merchant billing, customer payment, transaction states, refunds, reports, live activity, assistant behavior, and recovery steps.

1. Start here

Roles, services, sign-in, and the safest first run

System roles

Role	Primary work	Protected boundary
Merchant	Create/find bills, monitor customers and settlements, refund paid bills, export reports	Only the merchant's own bills, customers, transactions, and reports
Customer	View bills, pay pending bills, review payment history	Only the customer's own bills and transactions

First local run

1. Start PostgreSQL and confirm the mcs_db database exists.
2. Start Kafka on port 9092 when demonstrating the live feed.
3. Run the Spring Boot API on port 8080.
4. Run Flutter with MCS_API_BASE_URL pointing to the API.
5. Sign in as the intended role; merchant and customer sessions are deliberately separate.

Seeded demo access

Role	Email	Password
Merchant	sharma@electronics.com	McsDemo@123
Customer	rahul@gmail.com	McsDemo@123

Security

A 401 means the token is absent or expired. A 403 means the signed-in account does not own the requested resource. Do not work around either response by changing IDs.

2. Merchant workflow

From bill creation to customer visibility and settlement reporting

Create a bill

6. Open Bills and choose Create Bill.
7. Select the intended customer and enter an amount of at least ₹1.00.
8. Add a description that lets both parties identify the charge.
9. Submit once. The new bill appears as PENDING.

Find a bill without long scrolling

- Search by bill ID, customer, or description.
- Filter by Pending, Paid, Failed, or Refunded; combine with the date filter.
- Sort newest/oldest as needed. Clear filters to restore the full directory.

Connected customers

The Customers section is live backend data aggregated from the merchant's persisted bills. A customer becomes connected when the merchant has bill activity with that customer. Paid value and outstanding value are calculated from bill status, not typed into the dashboard.

Live transaction feed

- Settled payments publish a Kafka transaction-completed event.
- The dashboard receives the merchant's activity through the WebSocket feed.
- Use the horizontal/contained scrolling area for additional cards; it is intentionally not a tall page list.

3. Customer workflow

Review, pay, verify, and recover

Understand the dashboard

Area	Meaning
Outstanding	Total value of currently pending bills
Bills	Actual bill count and status distribution
Spending trend	Settled payment values by real activity date; labels/tooltips expose amounts
Recent activity	Latest persisted bills/payments, not mock examples
Next payment	The nearest actionable pending bill, or an all-caught-up state

Pay a bill

10. Open Bills, filter to Pending, and select the correct bill.
11. Review merchant, description, and amount before continuing.
12. Choose UPI, Card, or Net Banking and initiate the transaction.
13. The API returns an authorization decision. A decline leaves the bill pending for a safe retry.
14. After authorization, settlement changes the bill to PAID and creates a unique reference.

Verify the result

- The success receipt must show Transaction ID, settlement reference, merchant, amount, and PAID status.
- Payment History can be searched and filtered; use it instead of relying only on the dashboard card.

Never double-pay

If the screen is interrupted after authorization, reopen the bill and inspect its status/history before attempting another payment.

4. Lifecycle, refunds, and reports

Status meanings and operator rules

Transaction states

State	Meaning	Next valid action
INITIATED	Payment attempt recorded	Authorize
AUTHORIZED	Payment method approved	Settle
SETTLED	Funds recorded and settlement persisted	Report or refund the paid bill
FAILED	Authorization declined	Retry from the pending bill

Refund a payment

15. Merchant opens a bill with PAID status.
16. Choose Refund and enter a concise reason.
17. Confirm once. The backend records REFUNDED, the reason, and the refund timestamp.
18. Download the refund CSV from Reports when evidence is required.

Refund rule

Pending and failed bills cannot be refunded because no successful settlement exists.

Report outputs

Output	Contents	Use
Daily report	Today's settlement totals and counts	Daily reconciliation
Weekly report	Seven-day settlement totals and counts	Operational review
Bill statement CSV	Merchant bill directory	Spreadsheet analysis/audit
Refund CSV	Only persisted REFUNDED bills	Refund control evidence

5. Assistant and operational recovery

What is safe, what is live, and what to do when a service is down

Assistant behavior

- The assistant greets the signed-in merchant or customer once per login session.
- It summarizes account activity through role-aware backend logic and optional classifier support.
- It does not claim to create bills inside chat unless a real bill-creation action is implemented and confirmed.
- Clear removes the current conversation display; it does not delete bills, payments, or account data.

Recovery matrix

Symptom	Likely cause	Action
Customer service unavailable / 404	Old backend or wrong base URL	Start the current API and verify /customers/merchant/{merchantId}
Live feed stays still	Kafka/WebSocket unavailable or no new settlements	Start Kafka, reconnect, then settle a new test bill
Refund report empty	No REFUNDED bills	Refund one PAID bill, then export again
401	Missing/expired JWT	Sign in again
403	Wrong role or resource owner	Use the correct account; do not substitute IDs
Physical device cannot reach API	localhost points at the device	Pass the computer IP through MCS_API_BASE_URL

Data integrity checks

- One successful transaction must create exactly one settlement.
- A failed authorization must not create a settlement or mark the bill paid.
- A refund must preserve the original transaction/settlement evidence and update the bill to REFUNDED.

6. Acceptance and support reference

How to prove the build and hand it over

Automated evidence

Check	Command	Expected
Backend suite	.\mvnw.cmd test	Context, service, lifecycle, and security tests pass
Flutter suite	flutter test	Payment result and role registration tests pass without overflow
1,000 transactions	.\mvnw.cmd --% -q -Dtest=TransactionScalabilityTest -Dmcs.performance.tests=true test	1,000 settled; 0 failures; p95 under 2 s
Postman	Run MCS BRD API Acceptance	All executed requests under 2 s; no 5xx

Measured scalability result

Merchants	Transactions	REST calls	Failures	p95
100	1,000	3,000	0	9.45 ms

Environment: Spring Boot application context, JWT/MockMvc REST boundary, JPA, and H2 PostgreSQL mode. Kafka/network capacity is excluded; repeat against deployed PostgreSQL/Kafka before making a production infrastructure claim.

Handover artifacts

- README.md — setup and operating instructions
- postman/ — chained API acceptance collection and local environment
- docs/MCS_ER_Diagram.svg — database relationships
- docs/performance/1000-transactions-report.md — measured benchmark
- docs/MCS_Final_Demo.pptx — final presentation